# Divide-and-Conquer: Mergesort

## Design and Analysis of Algorithms

Dhannya SM

# Divide-and-Conquer

- An important programming paradigm
- Used in many classic algorithms
- Well known examples are binary search, mergesort

# Mergesort

- A classic example of divide and conquer technique
- Recursively divide the input list into two (nearly) equal sublists
- Sort each sublist individually
- Repeatedly merge the sorted sublists

# Merge

- Input: Two sorted lists A1 and A2

| | | | | |
|---|---|---|---|---|
| A1 | 10 | 9 | 6 | 2 |

| | | | | |
|---|---|---|---|---|
| A2 | 12 | 7 | 4 | 3 |

# Merge

- Compare the first two elements and copy the smallest to a new list, in this case 2 from A1

| | | | | |
|---|---|---|---|---|
| A1 | 10 | 9 | 6 | 2 |

| | | | | |
|---|---|---|---|---|
| A2 | 12 | 7 | 4 | 3 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | | | 2 |

# Merge

- Now, compare the second element in A1 and the first element in A2, that is the first two that have not yet been copied to the new list, and copy the smallest to the new list

| | | | | |
|---|---|---|---|---|
| A1 | 10 | 9 | 6 | 2 |

| | | | | |
|---|---|---|---|---|
| A2 | 12 | 7 | 4 | 3 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | | 3 | 2 |

# Merge

- Compare the next two elements . . .

| | | | | |
|---|---|---|---|---|
| A1 | 10 | 9 | 6 | 2 |

| | | | | |
|---|---|---|---|---|
| A2 | 12 | 7 | 4 | 3 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | 4 | 3 | 2 |

# Merge

- . . . and the next two . . .

| | | | | |
|---|---|---|---|---|
| A1 | 10 | 9 | 6 | 2 |

| | | | | |
|---|---|---|---|---|
| A2 | 12 | 7 | 4 | 3 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | 6 | 4 | 3 | 2 |

# Merge

- . . . and so on . . . until all elements are copied to the new list

| | | | | |
|---|---|---|---|---|
| A1 | 10 | 9 | 6 | 2 |

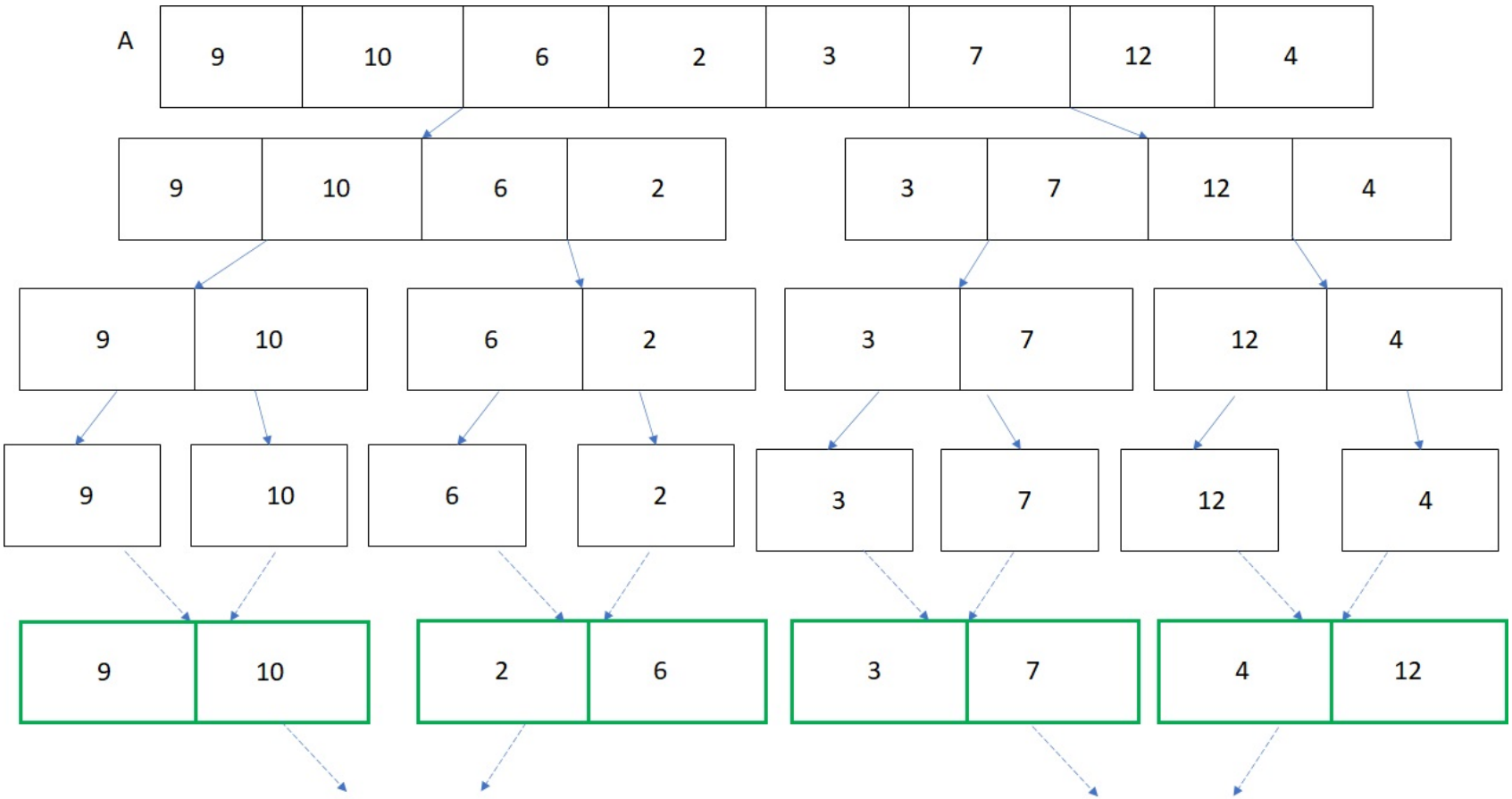
| | | | | |
|---|---|---|---|---|
| A2 | 12 | 7 | 4 | 3 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 12 | 10 | 9 | 7 | 6 | 4 | 3 | 2 |

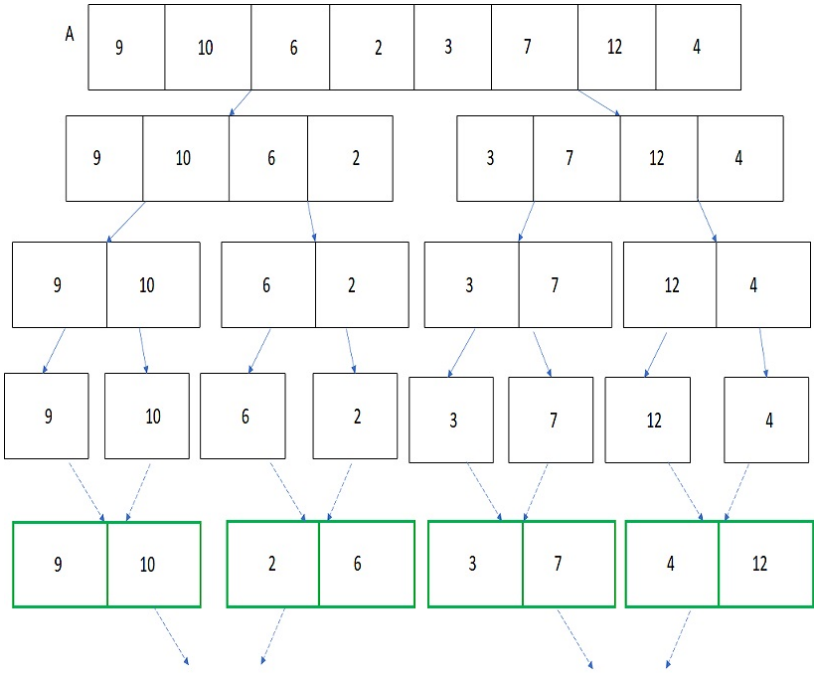
# Mergesort Algorithm

- Where do we get the two sorted lists A1 and A2 from?
- Recursively split the given list into sublists until each is of size 1
- Then, the first merge will simply compare two elements and copy them in sorted order to a new list of size 2.
- This goes on until the final sorted list is obtained.

# Mergesort Example

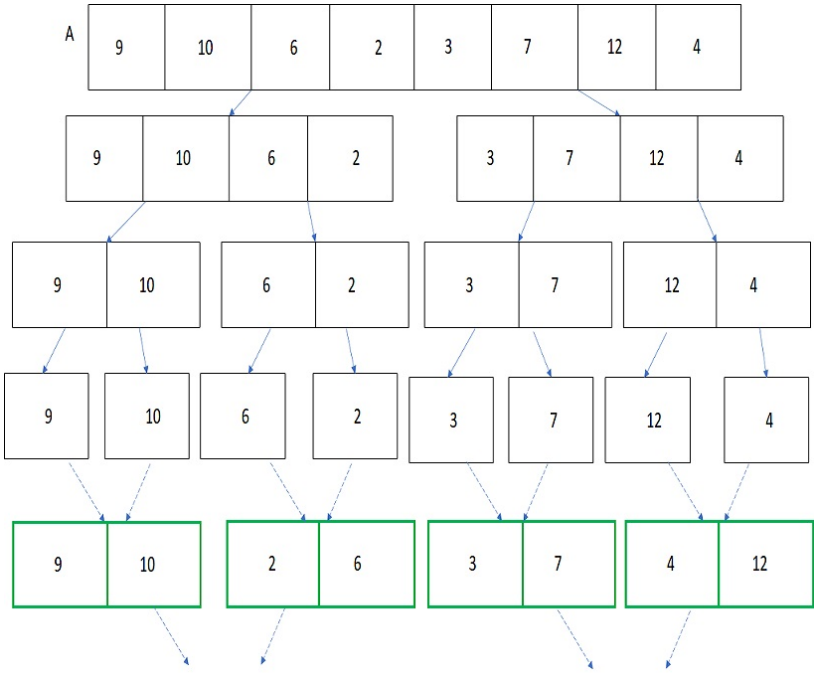


# Mergesort Algorithm



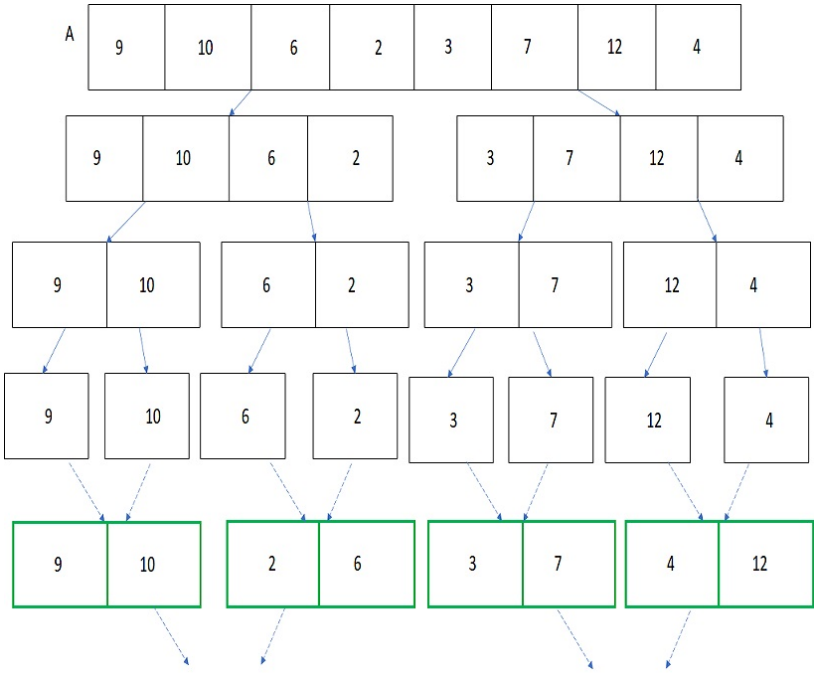

```
mergesort(A,low,high){
    ...
}
```

# Mergesort Algorithm



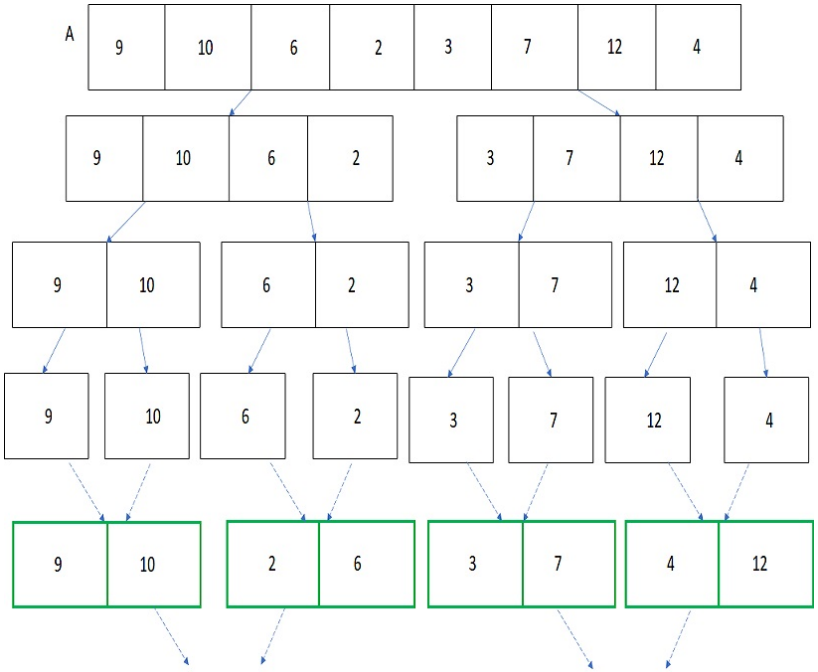

```
mergesort(A,low,high){
    if (low < high)
        mid = (low + high) / 2
    ...
}
```

# Mergesort Algorithm



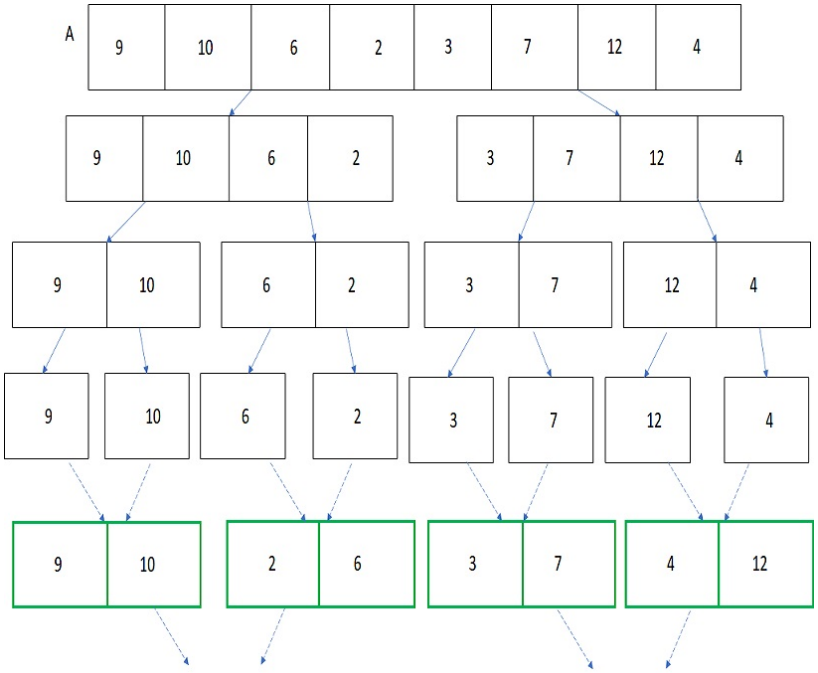

```
mergesort(A,low,high){
    if (low < high)
        mid = (low + high) / 2
    mergesort(A,low,mid)
    ...
}
```

# Mergesort Algorithm




```
mergesort(A,low,high){
    if (low < high)
        mid = (low + high) / 2
    mid = (low + high) / 2
    mergesort(A,low,mid)
    mergesort(A,mid+1,high)
    ...
}
```

# Mergesort Algorithm




```
mergesort(A,low,high){
    if (low < high)
        mid = (low + high) / 2
    mergesort(A,low,mid)
    mergesort(A,mid+1,high)
    merge(A,low,mid,high)
}
```

# Merge Algorithm

Try `merge` method as an exercise

```
merge(A,low,mid,high)
    ...
}
```

# Mergesort Analysis - Time complexity

Recurrence relation:

$$
\begin{aligned}
T(n) &= c_1 + T(n/2) + T(n/2) + c_2.n \\
&= 2T(n/2) + c_1 + c_2.n \\
&= 2T(n/2) + \Theta(n)
\end{aligned}
$$

Solving the recurrence, we get

$$T(n) = \Theta(n \ log \ n)$$

```
mergesort(A,low,high){
    if (low < high)
        mid = (low + high)/2
    mergesort(A,low,mid)
    mergesort(A,mid+1,high)
    merge(A,low,mid,high)
}
```

# Mergesort Analysis - Space complexity

- Is the mergesort in-place?

# Mergesort Analysis - Space complexity

- Is the mergesort in-place?
  - No, it uses additional $\Theta(n)$ space for the new list
- Is the mergesort stable?
  - Yes, why?

# Summary

- Uses divide and conquer strategy
- Repeatedly divides the list into sublists until each list is of size 1
- Merge the sublists by comparing the not-yet-copied elements into a sorted list
- Time complexity: $\Theta(n \ log \ n)$
- Space complexity: Additional space: $\Theta(n)$
- Not in-place
- Stable